# ECE 6140 Project 2: Deductive Fault Simulator

## 1. Implementation

The simulator is written in Python. It reads each netlist (gate lines `GATE in1 … out`, and `INPUT`/`OUTPUT` lines ending in `-1`), sorts the gates into evaluation order (the files are not ordered), and assigns vector bits to the primary inputs in `INPUT`-line order.

**Fault model.** Every net (primary inputs and all gate outputs) has stuck-at-0 and stuck-at-1, so there are 2 × (number of nets) faults. Fanout branches are not treated separately, as specified.

**Algorithm.** For one vector, the circuit is evaluated in level order. Each net *x* with fault-free value *v* carries a fault list *L(x)*, the set of faults that make *x* differ from *v*. A primary input *x* starts with *L(x)* = {x stuck-at-(1−v)}. Each gate computes the output list from its input lists, then adds its own output fault {out stuck-at-(1−v_out)}:

| Gate case | Output list |
|---|---|
| NOT / BUFF | L(in) |
| AND, NAND, OR, NOR with at least one controlling input (0 for AND/NAND, 1 for OR/NOR) | intersection of L over the controlling inputs, minus the union of L over the non-controlling inputs |
| AND, NAND, OR, NOR with no controlling input | union of L over all inputs |
| XOR / XNOR | symmetric difference of the input lists |

The detected faults for a vector are the union of the lists at all primary outputs. Lists are stored as Python sets of (net, value) pairs.

**Options.** (a) By default all 2 × nets faults are simulated. (b) With a fault file (lines `net value`), the detected set is intersected with the faults in the file. The output file lists the detected faults (`net stuck at value`) and their count.

**Validation.** The detected-fault lists match the course sample output. For all eight assignment vectors they also agree with an independent serial fault simulator and a bit-parallel simulator.

**Usage.** `python deductive_fsim.py CIRCUIT --vector BITS [--faults FILE] [--out FILE]`; `--random N --seed S --plot FILE` for Part (b); `--assignment DIR` runs all eight Part (a) cases.

## 2. Part (a): faults detected by the given vectors

| Circuit | Vector | Total faults | Detected | Coverage |
|---|---|---|---|---|
| s27 | 1101101 | 40 | 9 | 22.5% |
| s27 | 0101001 | 40 | 13 | 32.5% |
| s298f_2 | 10101011110010101 | 404 | 87 | 21.5% |
| s298f_2 | 11101110101110111 | 404 | 53 | 13.1% |
| s344f_2 | 101010101010111101111111 | 380 | 82 | 21.6% |
| s344f_2 | 111010111010101010001100 | 380 | 132 | 34.7% |
| s349f_2 | 101000000010101011111111 | 378 | 97 | 25.7% |
| s349f_2 | 111111101010101010001111 | 378 | 137 | 36.2% |

The complete lists of detected faults are in Appendix B (the eight output files).

## 3. Part (b): random vectors

Random vectors were applied one at a time, accumulating the detected faults; coverage = detected / (2 × nets) was recorded after each vector (2,000 vectors, seed 1). The 75% and 90% thresholds were also averaged over 20 seeds.

| Circuit | Faults | First vector with >75% (seed 1) | First vector with >90% (seed 1) | >75%, mean (max) of 20 runs | >90%, mean (max) of 20 runs | Final coverage (2,000 vectors) |
|---|---|---|---|---|---|---|
| s27 | 40 | 4 | 21 | 9.9 (19) | 19.6 (57) | 100% |
| s298f_2 | 404 | 15 | 42 | 17.0 (25) | 49.6 (94) | 100% |
| s344f_2 | 380 | 10 | 19 | 8.9 (16) | 22.4 (41) | 100% |
| s349f_2 | 378 | 10 | 19 | 8.9 (16) | 22.4 (41) | 99.7% |

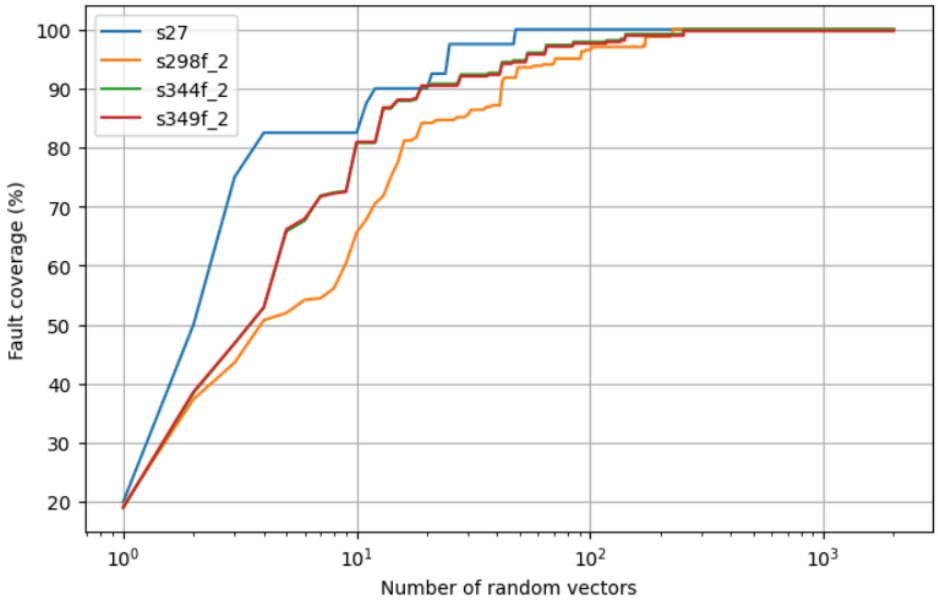



Figure 1. Simulator output, seed 1; first 250 of 2,000 random vectors shown

**Answer (seed 1).** More than 75% coverage needs 4, 15, 10 and 10 vectors for s27, s298f_2, s344f_2 and s349f_2 respectively; more than 90% needs 21, 42, 19 and 19. So at most 15 vectors reach 75% and at most 42 reach 90%. Averaged over 20 seeds the counts are 10, 17, 9, 9 (75%) and 20, 50, 22, 22 (90%).

**Observations.** Coverage rises steeply over the first 10 to 20 vectors, then flattens because the remaining faults need specific input combinations. s298f_2 is the slowest circuit and s27 the fastest; s344f_2 and s349f_2 behave almost identically. s349f_2 ends at 99.7%: net 179 stuck-at-1 was not detected even in 100,000 random vectors, so it is likely redundant. The other circuits reach 100%.

## Appendix A: Source code (deductive_fsim.py)

```
import argparse
import os
import random
import sys

CONTROLLING = {'AND': 0, 'NAND': 0, 'OR': 1, 'NOR': 1}
ALIAS = {'INV': 'NOT', 'BUF': 'BUFF'}

ASSIGNMENT_VECTORS = {
    's27':     ['1101101', '0101001'],
    's298f_2': ['10101011110010101', '11101110101110111'],
    's344f_2': ['10101010101011101111111', '11101011101010100010100'],
    's349f_2': ['10100000001010101111111', '11111110101010100010111'],
}


def parse_netlist(path):
    pis, pos, gates = [], [], []
    with open(path) as f:
        for line in f:
            t = [x for x in line.upper().split() if x != '-1']
            if not t:
                continue
            if t[0] == 'INPUT':
                pis += t[1:]
            elif t[0] == 'OUTPUT':
                pos += t[1:]
            elif len(t) >= 3:
                gates.append((ALIAS.get(t[0], t[0]), t[-1], t[1:-1]))
            else:
                raise ValueError(f'{path}: cannot parse line: {line.strip()}')
    known, order, pending = set(pis), [], gates
    while pending:
        ready = [g for g in pending if all(i in known for i in g[2])]
        if not ready:
            raise ValueError(f'{path}: undriven net or combinational loop near {pending[:3]}')
        known |= {g[1] for g in ready}
        order += ready
        pending = [g for g in pending if g not in ready]
    return {'name': os.path.basename(path), 'PI': pis, 'PO': pos, 'gates': order,
            'nets': set(pis) | {g[1] for g in order}}


def all_faults(c):
    return {(n, s) for n in c['nets'] for s in (0, 1)}


def read_fault_file(path, c):
    faults = set()
    with open(path) as f:
        for k, line in enumerate(f, 1):
            t = line.split('#')[0].split()
            if not t:
                continue
            if len(t) != 2 or t[1] not in ('0', '1') or t[0] not in c['nets']:
                print(f'warning: {path} line {k} ignored: {line.strip()!r}', file=sys.stderr)
                continue
            faults.add((t[0], int(t[1])))
    return faults


def gate_value(g, v):
    if g == 'AND':  return int(all(v))
    if g == 'NAND': return 1 - int(all(v))
    if g == 'OR':   return int(any(v))
    if g == 'NOR':  return 1 - int(any(v))
    if g == 'NOT':  return 1 - v[0]
    if g == 'BUFF': return v[0]
    p = sum(v) % 2
    if g == 'XOR':  return p
    if g == 'XNOR': return 1 - p
    raise ValueError(f'unknown gate type {g}')
def propagate(g, vals, lists):
    if g in CONTROLLING:
```

```
        c = CONTROLLING[g]
        at_c = [i for i, x in enumerate(vals) if x == c]
        if at_c:
            out = set.intersection(*[lists[i] for i in at_c])
            for i, x in enumerate(vals):
                if x != c:
                    out = out - lists[i]
            return out
        return set().union(*lists)
    if g in ('XOR', 'XNOR'):
        out = set()
        for l in lists:
            out = out ^ l
        return out
    return set(lists[0])


def simulate_vector(c, vec, targets=None):
    if len(vec) != len(c['PI']):
        raise ValueError(f"{c['name']}: vector has {len(vec)} bits but circuit has {len(c['PI'])} inputs")
    val, flist = {}, {}
    for n, b in zip(c['PI'], vec):
        val[n] = b
        flist[n] = {(n, 1 - b)}
    for g, out, ins in c['gates']:
        v = [val[i] for i in ins]
        val[out] = gate_value(g, v)
        flist[out] = propagate(g, v, [flist[i] for i in ins]) | {(out, 1 - val[out])}
    detected = set().union(*[flist[o] for o in c['PO']])
    return detected & targets if targets is not None else detected


def parse_vector(s):
    s = s.replace(' ', '')
    if not s or any(ch not in '01' for ch in s):
        raise ValueError(f'bad vector: {s!r}')
    return [int(ch) for ch in s]


def fault_key(f):
    return (int(f[0]) if f[0].isdigit() else 0, f[0], f[1])


def write_report(path, circuit, vectors, detected, total):
    with open(path, 'w') as f:
        f.write(f'Circuit: {circuit}   Input Vector(s): {"  ".join(vectors)}\n')
        f.write('------FAULTS DETECTED------\n')
        for n, s in sorted(detected, key=fault_key):
            f.write(f'{n:>5} stuck at {s}\n')
        f.write(f'\nNumber of faults detected: {len(detected)}\n')
        f.write(f'Number of faults simulated: {total}\n')
        f.write(f'Fault coverage: {100.0 * len(detected) / total:.2f}%\n')


def run_vectors(c, vectors, targets, out_path):
    total_set = targets if targets is not None else all_faults(c)
    detected = set()
    for v in vectors:
        detected |= simulate_vector(c, parse_vector(v), targets)
    write_report(out_path, c['name'], vectors, detected, len(total_set))
    print(f"{c['name']}  vector(s) {' '.join(vectors)}: {len(detected)} of {len(total_set)} faults detected "
          f"({100.0 * len(detected) / len(total_set):.1f}%)  ->  {out_path}")
    return detected


def random_coverage(c, n_vectors, seed, targets=None):
    rnd = random.Random(seed)
    total = len(targets) if targets is not None else 2 * len(c['nets'])
    detected, cov = set(), []
    for _ in range(n_vectors):
        vec = [rnd.randint(0, 1) for _ in c['PI']]
        detected |= simulate_vector(c, vec, targets)
        cov.append(100.0 * len(detected) / total)
    return cov


def first_above(cov, pct):
```

```
    return next((k + 1 for k, x in enumerate(cov) if x > pct), None)


def do_random(c, args, targets):
    cov = random_coverage(c, args.random, args.seed, targets)
    print(f"{c['name']}: {len(cov)} random vectors (seed {args.seed}), final coverage {cov[-1]:.2f}%")
    for pct in (75, 90):
        k = first_above(cov, pct)
        print(f"  >{pct}% coverage first reached at vector {k}" if k else f"  >{pct}% coverage not reached")
    if args.csv:
        with open(args.csv, 'w') as f:
            f.write('vectors,coverage_percent\n')
            for k, x in enumerate(cov, 1):
                f.write(f'{k},{x:.4f}\n')
    if args.plot:
        import matplotlib
        matplotlib.use('Agg')
        import matplotlib.pyplot as plt
        plt.figure(figsize=(7, 4.5))
        plt.plot(range(1, len(cov) + 1), cov, label=c['name'])
        for pct in (75, 90):
            plt.axhline(pct, color='gray', ls='--', lw=0.8)
        plt.xscale('log'); plt.xlabel('Number of random vectors'); plt.ylabel('Fault coverage (%)')
        plt.grid(True); plt.legend(); plt.tight_layout(); plt.savefig(args.plot, dpi=200)
        print(f'  plot saved to {args.plot}')


def main():
    ap = argparse.ArgumentParser(description='Deductive fault simulator (ECE 6140 Project 2)')
    ap.add_argument('circuit', nargs='?', help='netlist file')
    ap.add_argument('--vector', action='append', default=[], help='input vector, e.g. 1101101 (repeatable)')
    ap.add_argument('--vectors-file', help='file with one vector per line')
    ap.add_argument('--faults', help='option (b): fault list file ("net value" per line)')
    ap.add_argument('--out', help='output file (default <circuit>_out.txt)')
    ap.add_argument('--random', type=int, metavar='N', help='apply N random vectors and report coverage')
    ap.add_argument('--seed', type=int, default=1)
    ap.add_argument('--plot', help='save coverage plot (PNG) for --random')
    ap.add_argument('--csv', help='save coverage vs vectors (CSV) for --random')
    ap.add_argument('--assignment', metavar='DIR', help='run all 8 Part (a) vectors on the netlists in DIR')
    args = ap.parse_args()

    if args.assignment:
        for name, vecs in ASSIGNMENT_VECTORS.items():
            c = parse_netlist(os.path.join(args.assignment, name + '.txt'))
            for i, v in enumerate(vecs, 1):
                run_vectors(c, [v], None, os.path.join(args.assignment, f'{name}_v{i}_out.txt'))
        return

    if not args.circuit:
        ap.error('circuit file required')
    c = parse_netlist(args.circuit)
    targets = read_fault_file(args.faults, c) if args.faults else None

    if args.random:
        do_random(c, args, targets)
        return

    vectors = list(args.vector)
    if args.vectors_file:
        with open(args.vectors_file) as f:
            vectors += [l.strip() for l in f if l.strip()]
    if not vectors:
        ap.error('give --vector, --vectors-file, or --random')
    out = args.out or os.path.splitext(args.circuit)[0] + '_out.txt'
    run_vectors(c, vectors, targets, out)


if __name__ == '__main__':
    main()
```

## Appendix B: Output files

Each block is the content of one output file produced by `deductive_fsim.py --assignment`: the detected faults (net stuck at value) followed by the counts. Faults are shown in four columns to save space.

### s27_v1_out.txt

```
Circuit: s27.txt   Input Vector(s): 1101101
------FAULTS DETECTED------
     1 stuck at 0             7 stuck at 0            12 stuck at 0
     3 stuck at 1             9 stuck at 1            13 stuck at 0
     5 stuck at 0            11 stuck at 0            15 stuck at 1

Number of faults detected: 9
Number of faults simulated: 40
Fault coverage: 22.50%
```

### s27_v2_out.txt

```
Circuit: s27.txt   Input Vector(s): 0101001
------FAULTS DETECTED------
     1 stuck at 1             8 stuck at 1            14 stuck at 0            20 stuck at 1
     3 stuck at 1             9 stuck at 1            15 stuck at 1
     5 stuck at 0            11 stuck at 0            16 stuck at 1
     7 stuck at 1            12 stuck at 1            19 stuck at 1

Number of faults detected: 13
Number of faults simulated: 40
Fault coverage: 32.50%
```

### s298f_2_v1_out.txt

```
Circuit: s298f_2.txt   Input Vector(s): 10101011110010101
------FAULTS DETECTED------
     3 stuck at 0            30 stuck at 1            66 stuck at 0           138 stuck at 0
     5 stuck at 0            31 stuck at 1            67 stuck at 1           143 stuck at 1
     6 stuck at 1            32 stuck at 0            68 stuck at 1           144 stuck at 1
     7 stuck at 0            33 stuck at 1            95 stuck at 0           145 stuck at 1
     8 stuck at 0            34 stuck at 0           102 stuck at 0           149 stuck at 0
     9 stuck at 0            35 stuck at 0           103 stuck at 0           152 stuck at 0
    10 stuck at 0            36 stuck at 1           104 stuck at 0           158 stuck at 0
    11 stuck at 1            37 stuck at 0           107 stuck at 1           159 stuck at 0
    12 stuck at 1            39 stuck at 1           108 stuck at 1           161 stuck at 0
    15 stuck at 0            41 stuck at 1           109 stuck at 1           163 stuck at 0
    18 stuck at 1            45 stuck at 1           110 stuck at 1           166 stuck at 1
    19 stuck at 1            48 stuck at 0           111 stuck at 1           167 stuck at 1
    20 stuck at 1            49 stuck at 0           113 stuck at 1           168 stuck at 1
    21 stuck at 1            50 stuck at 0           115 stuck at 1           169 stuck at 1
    22 stuck at 1            51 stuck at 0           116 stuck at 1           170 stuck at 1
    23 stuck at 1            52 stuck at 1           118 stuck at 0           173 stuck at 0
    24 stuck at 0            53 stuck at 1           119 stuck at 0           182 stuck at 1
    25 stuck at 0            54 stuck at 1           121 stuck at 0           183 stuck at 1
    26 stuck at 0            55 stuck at 1           122 stuck at 0           186 stuck at 1
    27 stuck at 0            56 stuck at 0           132 stuck at 0           187 stuck at 1
    28 stuck at 1            58 stuck at 0           133 stuck at 0           188 stuck at 1
    29 stuck at 1            64 stuck at 1           135 stuck at 0

Number of faults detected: 87
Number of faults simulated: 404
Fault coverage: 21.53%
```

### s298f_2_v2_out.txt

```
Circuit: s298f_2.txt   Input Vector(s): 11101110101110111
------FAULTS DETECTED------
     1 stuck at 0            23 stuck at 1            37 stuck at 1            67 stuck at 1
     6 stuck at 0            24 stuck at 1            43 stuck at 0            68 stuck at 1
     7 stuck at 0            25 stuck at 1            45 stuck at 1           102 stuck at 1
     8 stuck at 1            26 stuck at 1            47 stuck at 1           152 stuck at 1
     9 stuck at 0            27 stuck at 1            49 stuck at 1           166 stuck at 0
    10 stuck at 1            28 stuck at 1            51 stuck at 1           168 stuck at 0
    11 stuck at 0            29 stuck at 1            53 stuck at 0           170 stuck at 0
    12 stuck at 0            30 stuck at 1            55 stuck at 0           172 stuck at 0
    15 stuck at 0            31 stuck at 1            57 stuck at 0           173 stuck at 0
    18 stuck at 1            32 stuck at 0            59 stuck at 0           174 stuck at 0
    19 stuck at 1            33 stuck at 0            63 stuck at 1           182 stuck at 0
    20 stuck at 1            34 stuck at 0            64 stuck at 0
    21 stuck at 1            35 stuck at 1            65 stuck at 1
    22 stuck at 1            36 stuck at 0            66 stuck at 1

Number of faults detected: 53
Number of faults simulated: 404
Fault coverage: 13.12%
```

### s344f_2_v1_out.txt

```
Circuit: s344f_2.txt   Input Vector(s): 101010101010111101111111
------FAULTS DETECTED------
     1 stuck at 0            30 stuck at 0            54 stuck at 1            97 stuck at 0
     2 stuck at 1            31 stuck at 0            60 stuck at 0            99 stuck at 0
     3 stuck at 0            32 stuck at 1            61 stuck at 1           101 stuck at 0
     4 stuck at 1            33 stuck at 0            62 stuck at 1           105 stuck at 1
     5 stuck at 0            34 stuck at 1            64 stuck at 0           106 stuck at 0
     6 stuck at 1            35 stuck at 0            65 stuck at 1           108 stuck at 0
     7 stuck at 0            36 stuck at 1            66 stuck at 1           109 stuck at 1
     8 stuck at 1            37 stuck at 0            67 stuck at 0           110 stuck at 1
     9 stuck at 0            38 stuck at 0            68 stuck at 1           111 stuck at 1
    10 stuck at 1            39 stuck at 0            69 stuck at 1           112 stuck at 0
    11 stuck at 0            40 stuck at 1            70 stuck at 1           114 stuck at 0
    12 stuck at 1            41 stuck at 0            71 stuck at 1           115 stuck at 1
    13 stuck at 0            42 stuck at 1            72 stuck at 1           116 stuck at 1
    14 stuck at 0            43 stuck at 0            73 stuck at 1           144 stuck at 0
    15 stuck at 0            44 stuck at 1            76 stuck at 1           145 stuck at 0
    16 stuck at 0            45 stuck at 0            77 stuck at 1           146 stuck at 0
    25 stuck at 1            46 stuck at 1            78 stuck at 1           188 stuck at 1
    26 stuck at 1            47 stuck at 0            91 stuck at 0           189 stuck at 1
    27 stuck at 1            48 stuck at 0            92 stuck at 1           190 stuck at 1
    28 stuck at 0            49 stuck at 1            95 stuck at 1
    29 stuck at 0            50 stuck at 0            96 stuck at 1

Number of faults detected: 82
Number of faults simulated: 380
Fault coverage: 21.58%
```

**s344f_2_v2_out.txt**

```
Circuit: s344f_2.txt   Input Vector(s): 111010111010101010001100
------FAULTS DETECTED------
     1 stuck at 0            42 stuck at 1            80 stuck at 0           136 stuck at 0
     2 stuck at 0            43 stuck at 0            82 stuck at 0           137 stuck at 1
     3 stuck at 0            44 stuck at 1            85 stuck at 0           138 stuck at 1
     4 stuck at 1            45 stuck at 0            86 stuck at 0           139 stuck at 1
     5 stuck at 0            46 stuck at 1            88 stuck at 0           140 stuck at 1
     6 stuck at 1            47 stuck at 1            91 stuck at 1           142 stuck at 0
     7 stuck at 0            48 stuck at 1            92 stuck at 0           144 stuck at 0
     8 stuck at 0            49 stuck at 0            93 stuck at 0           146 stuck at 0
     9 stuck at 0            50 stuck at 1            94 stuck at 0           147 stuck at 1
    10 stuck at 1            51 stuck at 0            95 stuck at 1           148 stuck at 1
    11 stuck at 0            52 stuck at 0            96 stuck at 1           149 stuck at 0
    12 stuck at 1            53 stuck at 1            97 stuck at 0           153 stuck at 1
    13 stuck at 0            55 stuck at 0            99 stuck at 1           154 stuck at 1
    14 stuck at 1            56 stuck at 1           100 stuck at 1           155 stuck at 0
    15 stuck at 0            57 stuck at 0           101 stuck at 0           156 stuck at 0
    16 stuck at 1            58 stuck at 1           103 stuck at 0           162 stuck at 1
    25 stuck at 1            59 stuck at 1           104 stuck at 1           163 stuck at 1
    26 stuck at 1            60 stuck at 1           117 stuck at 0           164 stuck at 1
    27 stuck at 1            61 stuck at 1           118 stuck at 1           165 stuck at 0
    28 stuck at 0            62 stuck at 1           119 stuck at 1           169 stuck at 1
    29 stuck at 1            63 stuck at 1           120 stuck at 1           170 stuck at 0
    30 stuck at 0            64 stuck at 1           122 stuck at 0           171 stuck at 0
    31 stuck at 1            65 stuck at 0           123 stuck at 0           173 stuck at 1
    32 stuck at 0            67 stuck at 0           124 stuck at 1           174 stuck at 1
    33 stuck at 0            68 stuck at 1           125 stuck at 1           175 stuck at 0
    34 stuck at 0            69 stuck at 0           126 stuck at 0           179 stuck at 0
    35 stuck at 1            70 stuck at 1           127 stuck at 1           181 stuck at 1
    36 stuck at 1            71 stuck at 0           128 stuck at 1           182 stuck at 1
    37 stuck at 0            72 stuck at 0           130 stuck at 0           183 stuck at 1
    38 stuck at 1            73 stuck at 1           131 stuck at 0           184 stuck at 1
    39 stuck at 0            76 stuck at 0           132 stuck at 1           186 stuck at 1
    40 stuck at 1            78 stuck at 0           133 stuck at 1           188 stuck at 0
    41 stuck at 0            79 stuck at 0           134 stuck at 1           189 stuck at 0

Number of faults detected: 132
Number of faults simulated: 380
Fault coverage: 34.74%
```

### s349f_2_v1_out.txt

```
Circuit: s349f_2.txt   Input Vector(s): 101000000010101011111111
------FAULTS DETECTED------
     1 stuck at 0            34 stuck at 1            60 stuck at 0           128 stuck at 1
     2 stuck at 1            35 stuck at 0            62 stuck at 0           129 stuck at 1
     3 stuck at 0            36 stuck at 1            64 stuck at 1           130 stuck at 1
     4 stuck at 1            37 stuck at 0            65 stuck at 1           131 stuck at 1
     5 stuck at 1            38 stuck at 1            66 stuck at 0           133 stuck at 0
     6 stuck at 1            39 stuck at 0            67 stuck at 1           134 stuck at 1
     7 stuck at 1            40 stuck at 1            68 stuck at 0           135 stuck at 1
     8 stuck at 1            41 stuck at 0            69 stuck at 1           137 stuck at 0
     9 stuck at 1            42 stuck at 0            70 stuck at 1           138 stuck at 1
    10 stuck at 1            43 stuck at 1            71 stuck at 1           171 stuck at 0
    11 stuck at 0            44 stuck at 0            72 stuck at 1           173 stuck at 0
    12 stuck at 1            45 stuck at 0            73 stuck at 1           174 stuck at 1
    13 stuck at 0            46 stuck at 0            74 stuck at 1           176 stuck at 0
    14 stuck at 1            47 stuck at 0           109 stuck at 0           177 stuck at 1
    15 stuck at 0            48 stuck at 0           111 stuck at 0           178 stuck at 1
    16 stuck at 1            49 stuck at 0           113 stuck at 0           179 stuck at 0
    25 stuck at 0            50 stuck at 0           115 stuck at 0           180 stuck at 0
    26 stuck at 1            51 stuck at 1           116 stuck at 1           181 stuck at 0
    27 stuck at 0            52 stuck at 0           118 stuck at 1           183 stuck at 0
    28 stuck at 1            53 stuck at 1           120 stuck at 0           187 stuck at 1
    29 stuck at 1            54 stuck at 1           121 stuck at 1           188 stuck at 1
    30 stuck at 1            55 stuck at 1           123 stuck at 0           189 stuck at 1
    31 stuck at 1            56 stuck at 1           124 stuck at 1
    32 stuck at 1            57 stuck at 1           126 stuck at 0
    33 stuck at 1            58 stuck at 0           127 stuck at 0

Number of faults detected: 97
Number of faults simulated: 378
Fault coverage: 25.66%
```

**s349f_2_v2_out.txt**

```
Circuit: s349f_2.txt   Input Vector(s): 111111101010101010001111
------FAULTS DETECTED------
     1 stuck at 0            45 stuck at 1            83 stuck at 1           138 stuck at 1
     2 stuck at 0            46 stuck at 0            84 stuck at 0           143 stuck at 1
     3 stuck at 0            47 stuck at 1            85 stuck at 1           144 stuck at 1
     4 stuck at 0            48 stuck at 1            86 stuck at 1           145 stuck at 1
     5 stuck at 0            49 stuck at 1            87 stuck at 0           150 stuck at 1
     6 stuck at 0            50 stuck at 1            88 stuck at 1           151 stuck at 1
     7 stuck at 0            51 stuck at 0            89 stuck at 1           152 stuck at 1
     8 stuck at 1            52 stuck at 0            90 stuck at 0           157 stuck at 1
     9 stuck at 0            53 stuck at 0            91 stuck at 1           158 stuck at 1
    10 stuck at 1            54 stuck at 0            92 stuck at 1           159 stuck at 1
    11 stuck at 0            55 stuck at 0            93 stuck at 0           162 stuck at 1
    12 stuck at 1            56 stuck at 0            94 stuck at 0           163 stuck at 1
    13 stuck at 0            57 stuck at 0            95 stuck at 0           164 stuck at 1
    14 stuck at 1            58 stuck at 1            96 stuck at 1           165 stuck at 1
    15 stuck at 0            59 stuck at 1            97 stuck at 0           166 stuck at 1
    25 stuck at 1            61 stuck at 0            99 stuck at 0           167 stuck at 0
    26 stuck at 1            62 stuck at 1           102 stuck at 0           168 stuck at 1
    27 stuck at 1            63 stuck at 1           103 stuck at 0           169 stuck at 0
    28 stuck at 0            65 stuck at 0           105 stuck at 0           171 stuck at 0
    29 stuck at 0            66 stuck at 0           108 stuck at 1           173 stuck at 0
    30 stuck at 0            67 stuck at 1           109 stuck at 1           174 stuck at 0
    31 stuck at 0            68 stuck at 0           110 stuck at 1           175 stuck at 0
    32 stuck at 0            69 stuck at 1           111 stuck at 1           177 stuck at 0
    33 stuck at 1            70 stuck at 0           112 stuck at 1           180 stuck at 1
    34 stuck at 0            71 stuck at 0           113 stuck at 1           181 stuck at 1
    35 stuck at 1            72 stuck at 0           114 stuck at 1           182 stuck at 1
    36 stuck at 1            74 stuck at 0           115 stuck at 1           183 stuck at 1
    37 stuck at 0            75 stuck at 0           116 stuck at 0           184 stuck at 1
    38 stuck at 1            76 stuck at 1           117 stuck at 0           185 stuck at 1
    39 stuck at 0            77 stuck at 1           130 stuck at 1           186 stuck at 1
    40 stuck at 0            78 stuck at 0           131 stuck at 1           187 stuck at 0
    41 stuck at 1            79 stuck at 1           133 stuck at 0           188 stuck at 0
    42 stuck at 1            80 stuck at 1           134 stuck at 1
    43 stuck at 1            81 stuck at 0           135 stuck at 1
    44 stuck at 0            82 stuck at 1           137 stuck at 0

Number of faults detected: 137
Number of faults simulated: 378
Fault coverage: 36.24%
```